

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

Кафедра програмних засобів

ЗВІТ

Дисципліна «Технології розподілених систем та паралельних обчислень»

Робота №2

Тема «Використання бібліотеки MPI для створення паралельних програм»

Виконав варіант 19

Студент КНТ-122

Онищенко О.А.

Прийняли

Викладач

Дьячук Т.С.

2025

МЕТА

Метою роботи є вивчити каркас паралельних програм на базі функцій бібліотеки стандарту MPI, навчитися створювати паралельні програми та запускати їх на виконання.

ЗАВДАННЯ

Завдання на роботу:

1. Змінити вихідну програму так, щоб кожний процес виводив інформацію щодо парності свого номера.

2. Доповніть вихідну програму так, щоб процес "1" виконував посилку, а процес "0" – прийом елементів масиву цілих чисел типу long, причому, розміри буферів передачі та прийому мають дорівнювати номеру робочого місця, помноженому на 10, а кількість елементів, що передаються – номеру робочого місця, збільшеного на одиницю. В процесі "0" реалізувати виведення кількості фактично прийнятих елементів.

3. Переробити програму завдання 2 так, щоб в якості буферів прийому та передачі використовувались масиви динамічної пам'яті.

4. Напишіть програму, яка складається з чотирьох процесів. Процес "0" передає до процесів "1", "2" та "3" рядок з прізвищем студента. Процес "1" конкатенує прийнятий рядок з рядком, відповідним імені студента, і відсилає отриманий рядок назад. Процес "2" визначає кількість символів в прийнятому рядку та відсилає це число до нульового процесу. Процес "3" множить число, що дорівнює сумі кодів символів прийнятого рядка на число PI, та відсилає отримане значення до процесу "0".

Після завершення обмінів процес "0" виводить на друк отримані від інших процесів значення.

ВИКОНАННЯ

1 Парність процесів

Завдання

Змінити вихідну програму так, щоб кожний процес виводив інформацію щодо парності свого номера.

Код

```
from mpi4py import MPI
import sys

def main():
    comm = MPI.COMM_WORLD
    size = comm.Get_size()
    rank = comm.Get_rank()

    parity = "even" if rank % 2 == 0 else "odd"
    print(f"Process {rank} of {size} is {parity}")

    comm.Barrier()

    if rank == 0:
        print(f"\nTotal processes: {size}")
        print(f"CommandLine for {rank} process:")
        for i, arg in enumerate(sys.argv):
            print(f"{i}: {arg}")

if __name__ == "__main__":
    main()
```

Результати

```
Process 1 of 2 is odd
Process 0 of 2 is even

Total processes: 2
CommandLine for 0 process:
0: 1.py
```

Рисунок 1.1 — Результати роботи

2 Доповнення

Завдання

Доповніть вихідну програму так, щоб процес "1" виконував посилку, а процес "0" – прийом елементів масиву цілих чисел типу long, причому, розміри буферів передачі та прийому мають дорівнювати номеру робочого місця, помноженому на 10, а кількість елементів, що передаються – номеру робочого місця, збільшеного на одиницю. В процесі "0" реалізувати виведення кількості фактично прийнятих елементів.

Код

```
from mpi4py import MPI
import numpy as np

def main():
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()
    size = comm.Get_size()

    # --- Налаштування згідно з варіантом ---
    WOKRSTATION_ID = 19

    buffer_size = WOKRSTATION_ID * 10
    elements_to_send = WOKRSTATION_ID + 1
    # --- † ---

    if size < 2:
        if rank == 0:
            print("At least 2 processes needed.")
            return

    if rank == 1:
        # Процес 1: Посилка
        # Створити масив даних типу Long (int64)
        data = np.arange(elements_to_send, dtype=np.int64) + 100
        print(f"Process {rank}: Sending {len(data)} elements: {data}")
        comm.Send([data, MPI.LONG], dest=0, tag=77)

    elif rank == 0:
        # Процес 0: Прийом
```



```

WOKRSTATION_ID = 19
buffer_size = WOKRSTATION_ID * 10
elements_to_send = WOKRSTATION_ID + 1
# --- + ---

if size < 2:
    if rank == 0:
        print("At least 2 processes needed.")
        return

if rank == 1:
    # Динамічне виділення пам'яті
    # Використання np.empty для виділення пам'яті
    send_data = np.empty(elements_to_send, dtype=np.int64)

    # Ініціалізація даних
    for i in range(elements_to_send):
        send_data[i] = i + 100

    print(
        f"Process {rank}: Dynamically created an array of
{elements_to_send} elements."
    )
    print(f"Process {rank}: Sending data: {send_data}")

    # Передача даних
    comm.Send([send_data, MPI.LONG], dest=0, tag=77)

elif rank == 0:
    # Динамічне всиділення пам'яті для буфера обміну
    # Виділяємо пам'яті під максимально можливий розмір буфера
    recv_buffer = np.empty(buffer_size, dtype=np.int64)

    status = MPI.Status()

    # Прийом даних у динамічно виділений буфер
    comm.Recv([recv_buffer, MPI.LONG], source=1, tag=77,
status=status)

    actual_count = status.Get_count(MPI.LONG)

    print(
        f"Process 0: Received data into a dynamic buffer (buffer
size: {buffer_size})"
    )
    print(f"Actual amount of received processes: {actual_count}")
    print(f"Contents of received data: {recv_buffer[:actual_count]}")

    # Явне звільнення пам'яті в Python відбувається автоматично (Garbage
Collector)

if __name__ == "__main__":
    main()

```

Результати

```
Process 1: Dynamically created an array of 20 elements.
Process 1: Sending data: [100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119]
Process 0: Received data into a dynamic buffer (buffer size: 190)
Actual amount of received processes: 40
Contents of received data: [
    103      104      105
    106      107      108
    109      110      111
    112      113      114
    115      116      117
    118      119 2334381307661018470
7161130726689436456 7381153676602075508 7813587163122463776
8317129717281943161 8029390568361167406 2314885436767102324
8245921732299941972 2338619869330699117 8007511636764026479
7960513042232403052 5782939046383283567 7305804419170315849
7957695011148363296 7526676251469033760 8603403747418180197
7885642940338368544 2326478923160641897 2334102031773885796
8101260421628980841]
```

Рисунок 3.1 — Результати роботи

4 Чотири процеси

Завдання

Напишіть програму, яка складається з чотирьох процесів. Процес "0" передає до процесів "1", "2" та "3" рядок з прізвищем студента. Процес "1" конкатенує прийнятий рядок з рядком, відповідним імені студента, і відсилає отриманий рядок назад. Процес "2" визначає кількість символів в прийнятому рядку та відсилає це число до нульового процесу. Процес "3" множить число, що дорівнює сумі кодів символів прийнятого рядка на число π , та відсилає отримане значення до процесу "0".

Після завершення обмінів процес "0" виводить на друк отримані від інших процесів значення.

Код

```
import math
from mpi4py import MPI
```

```

def main():
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()
    size = comm.Get_size()

    if size  $\neq$  4:
        if rank == 0:
            print(f"Error: This program needs 4 processes, but {size}
given.")
        return

    surname = "Onyshchenko"

    if rank == 0:
        print(f"[Process 0]: Sending a surname {surname} to processes 1,
2, and 3 ... ")

        for i in range(1, 4):
            comm.send(surname, dest=i)

        full_name = comm.recv(source=1)
        char_count = comm.recv(source=2)
        pi_result = comm.recv(source=3)

        print("--- Results for process 0 ---")
        print(f"From process 1 (Surname and Name): {full_name}")
        print(f"From process 2 (Number of characters): {char_count}")
        print(f"From process 3 (Sum of codes): {pi_result}")

    elif rank == 1:
        received_surname = comm.recv(source=0)
        name = "Oleh"
        result = f"{received_surname} {name}"
        comm.send(result, dest=0)

    elif rank == 2:
        received_surname = comm.recv(source=0)
        result = len(received_surname)
        comm.send(result, dest=0)

    elif rank == 3:
        received_surname = comm.recv(source=0)
        char_sum = sum(ord(char) for char in received_surname)
        result = char_sum * math.pi
        comm.send(result, dest=0)

if __name__ == "__main__":
    main()

```

Результати

```
[Process 0]: Sending a surname Onyshchenko to processes 1, 2, and 3...
--- Results for process 0 ---
From process 1 (Surname and Name): Onyshchenko Oleh
From process 2 (Number of characters): 11
From process 3 (Sum of codes): 3647.3890708177496
```

Рисунок 4.1 — Результати роботи

ПИТАННЯ

Який стандарт MPI та яку назву має бібліотека (реалізація) функцій MPI, які використовуються в даній лабораторній роботі?

Стандарт — OpenMPI.

Бібліотека — mpi4py (для Python, для C++ — mpi.h).

Які функції вихідної програми завдання 1 виконуються у всіх процесах?

Наступні функції виконуються у всіх процесах:

- MPI_Init;
- MPI_Comm_size;
- MPI_Comm_rank;
- MPI_Finalize.

Що означає ідентифікатор MPI_COMM_WORLD?

MPI_COMM_WORLD є комунікатором, який об'єднує всі процеси, запущені в межах однієї паралельної програми.

Джерело:

<https://www.dcs.ed.ac.uk/home/trollius/www.osc.edu/Lam/mpi/tutorial1.html#:>

[~:text=MPI_COMM_WORLD%20means%20%22all%20the%20processes%20in%20the%20MPI%20application.](#)

[%22%20It%20is%20called%20a%20communicator%20and%20it%20provides%20all%20information%20necessary%20to%20do%20message%20passing.](#)

Для чого використовується власний номер процесу в комунікаторі?

Без знання свого номера, кожен процес робив би те ж саме.

Напишіть фрагмент паралельної програми, який використовує значення кількості процесів в області зв'язку.

Код мовою Python з бібліотекою mpi4py наведено нижче:

```
from mpi4py import MPI

comm = MPI.COMM_WORLD

size = comm.Get_size()
rank = comm.Get_rank()

total_data_size = 1000

chunk_size = total_data_size // size
start_index = rank * chunk_size
end_index = (rank + 1) * chunk_size if rank != size - 1 else
total_data_size

print(
    f"[Процес {rank}/{size}]: Працюю з індексами від {start_index} до {end_index - 1}"
)

local_sum = sum(range(start_index, end_index))
total_sum = comm.reduce(local_sum, op=MPI.SUM, root=0)

if rank == 0:
    print(f"\nЗагальна сума, зібрана з {size} процесів: {total_sum}")
```

Результати роботи наведено на рисунку нижче:

```
[Process 4/5]: Working with indeces from 800 to 999
[Process 3/5]: Working with indeces from 600 to 799
[Process 2/5]: Working with indeces from 400 to 599
[Process 0/5]: Working with indeces from 0 to 199

Total sum, collected from 5 processes: 499500
[Process 1/5]: Working with indeces from 200 to 399
```

Рисунок 5.1 — Результати роботи

Напишіть фрагмент паралельної програми, в кожному з процесів якої створюється масив динамічної пам'яті, розмір якого дорівнює добутку номера процесу на загальну кількість процесів.

Написану програму наведено нижче:

```
from mpi4py import MPI
import numpy as np

comm = MPI.COMM_WORLD
size = comm.Get_size()
rank = comm.Get_rank()

array_size = rank * size

data_array = np.zeros(array_size, dtype=int)

print(f"Процес {rank}: Розмір масиву = {rank} * {size} = {array_size}")
print(f"Пам'ять виділено за адресою: {data_array.__array_interface__['data'][0]}")

if array_size > 0:
    print(f"\t→ Перші елементи процесу {rank}: {data_array[:5]} ... ")
else:
    print(f"\t→ Процес {rank} має порожній масив, (бо rank = 0)")
```

Результати роботи наведено нижче на рисунку:

```
Process 1: Array size = 1 * 5 = 5
Memory allocated from address: 1839984261520
    -> First process's elements 1: [0 0 0 0 0]...
Process 2: Array size = 2 * 5 = 10
Memory allocated from address: 2924413581168
    -> First process's elements 2: [0 0 0 0 0]...
Process 3: Array size = 3 * 5 = 15
Memory allocated from address: 2271492682864
    -> First process's elements 3: [0 0 0 0 0]...
Process 0: Array size = 0 * 5 = 0
Memory allocated from address: 1455968874352
    -> Process 0 has an empty array, (for, rank = 0))
Process 4: Array size = 4 * 5 = 20
Memory allocated from address: 1464299330272
    -> First process's elements 4: [0 0 0 0 0]...
```

Рисунок 5.2 — Результати виконання

**До якого класу належать програми даної лабораторної роботи:
SIMD чи MIMD?**

SIMD – Single Instruction Multiple Data. Всі паралельні блоки виконують ту ж інструкцію, але над різними елементами даних.

MIMD – Multiple Instruction Multiple Data. Всі паралельні блоки мають окремі інструкції і можуть виконувати їх в будь-який час.

Програми, створені впродовж цієї лабораторної роботи є типу SIMD.